

CloudPulse – Cloud Resource Monitoring Dashboard

Oct 3, 2026 · @Cecil

Cover page

A Project Report on

CloudPulse: Cloud Resource Monitoring Dashboard

Submitted in partial fulfilment of the requirements for the degree of **Bachelor of Science in Information Technology (B.Sc. IT)**

Submitted by	Cecil Baraik
Roll number	2025-BSCIT/004A
Programme	B.Sc. IT
Semester	----
Project guide	---- (designation: ----)
Institution	Amity University Jharkhand
Academic year	2026

Declaration

I, Cecil Baraik, declare that this project report titled "CloudPulse: Cloud Resource Monitoring Dashboard" is my own work, carried out under the guidance of ----, and has not been submitted elsewhere for any degree or diploma.

Signature: ---- Date: ----

Certificate

This is to certify that the project "CloudPulse: Cloud Resource Monitoring Dashboard" was carried out by Cecil Baraik (2025-BSCIT/004A) under my supervision, in partial fulfilment of the B.Sc. IT programme at Amity University Jharkhand.

Project guide: ---- Head of Department: ----

Acknowledgement

I thank my project guide _____ for guidance throughout this project, the faculty of the Department of Information Technology, Amity University Jharkhand, for their support, and my family and friends for their encouragement.

Abstract

CloudPulse is a web dashboard that monitors cloud infrastructure in real time. It tracks CPU, memory, disk and network usage for 11 resources: 10 simulated cloud servers on AWS, Azure and Google Cloud, and the real computer running the app, measured through the OSHI library.

The backend is written in Java 21 with Spring Boot 3.5 and stores data in MySQL 8. It collects metrics every 3 seconds and pushes them to the browser over WebSocket (STOMP), so charts update without refreshing the page. The frontend is a React 19 single-page app with animated charts, a dark and a light theme, and a mobile layout.

The system raises alerts when a resource breaks a configurable rule, emails critical alerts through Gmail, estimates monthly cost against a budget with saving suggestions, shows live logs, and exports a PDF report and CSV files. Access is protected with JWT login and two roles, Admin and Viewer.

Keywords: cloud monitoring, Spring Boot, React, WebSocket, MySQL, JWT, real-time dashboard

Chapter 1: Introduction

1.1 Background

Companies now run their applications on cloud servers from providers such as AWS, Microsoft Azure and Google Cloud. A single team may run dozens of virtual machines, databases, containers and serverless functions across several providers. If one of them runs out of CPU, memory or disk, the application slows down or stops, and every hour a server runs costs money.

Monitoring tools such as AWS CloudWatch, Datadog and Grafana show this information, but each provider has its own console and the professional tools are complex and paid. CloudPulse brings the key information into one simple, live dashboard.

1.2 Problem statement

An operations team needs to answer four questions quickly: Are all servers healthy right now? What happened in the last hours? Is anything about to fail? How much is all of this costing? Without a single dashboard, answering these means opening several consoles, refreshing pages by hand and noticing problems only after users complain.

1.3 Objectives

1. Show live CPU, memory, disk and network usage of every resource, updated every 3 seconds without page refresh.
2. Keep 24 hours of history and draw it as charts for 15 minutes, 1, 6 and 24 hours.
3. Raise alerts automatically when a value crosses a rule, and email critical alerts.
4. Let an admin start, stop and restart resources, while viewers have read-only access.
5. Estimate cost per hour and per month, compare it with a budget and suggest savings.
6. Show live logs from every resource with filters and search.
7. Export a PDF report and CSV files.
8. Store all data permanently in a relational database (MySQL).

1.4 Scope

The project monitors one real machine (the computer running the backend) and 10 simulated cloud resources whose metrics, costs and logs are generated to behave like real servers, including daily usage patterns and random load spikes. Connecting to real cloud provider APIs is outside the scope because it needs paid accounts; the design keeps this possible as future work (Chapter 8).

Chapter 2: Requirements and feasibility

2.1 Functional requirements

ID	Requirement
FR1	Users log in with username and password; the system issues a JWT token valid for 8 hours.
FR2	Two roles: Admin (full access) and Viewer (read-only).
FR3	Dashboard shows totals, average CPU and memory, network throughput, monthly cost and resource health, updated live.

ID	Requirement
FR4	Resource list with search, filter by provider, type and status, sorting, and card or table view.
FR5	Resource detail page with gauges and history charts (live, 15 min, 1 h, 6 h, 24 h).
FR6	Admin can start, stop and restart a resource.
FR7	Admin can create, edit, enable/disable and delete alert rules (metric, condition, threshold, severity, resource).
FR8	Alerts appear instantly as pop-ups; users can acknowledge one or all.
FR9	Critical alerts are emailed to up to 5 addresses, limited to 20 emails per hour.
FR10	Cost page: burn rate, projected month, last 24 h spend, budget bar, breakdowns and saving suggestions.
FR11	Live log viewer with level filter, resource filter, text search, pause and download.
FR12	Export a PDF summary report and CSV files of resources, alerts and costs.

2.2 Non-functional requirements

- **Performance:** a metrics update reaches the browser within one 3-second tick; charts are reduced to at most 90 points so they stay fast.
- **Security:** passwords hashed with BCrypt; every API call and WebSocket connection needs a valid JWT; admin-only actions checked on the server; the Gmail password is read from environment variables, never stored in code.
- **Reliability:** data survives restarts in MySQL; history older than 24 hours is cleaned automatically.
- **Usability:** dark and light theme, works on phone screens, status always shown with icon plus colour plus text.

2.3 Hardware and software requirements

Item	Used in this project
Processor / RAM	Any modern laptop, 8 GB RAM or more (developed on 16 GB)
Operating system	Windows 11 (also runs on Linux and macOS)
Java	JDK 21 (Eclipse Temurin)
Node.js / npm	Node 24, npm 11
Database	MySQL Community Server 8.0
IDE	IntelliJ IDEA Community 2025.3
Build tools	Maven (backend), Vite (frontend)
Browser	Google Chrome or any modern browser
Version control	Git and GitHub

2.4 Feasibility

- **Technical:** all tools are free and open source and run on a student laptop.
- **Economic:** no cost; simulated cloud resources avoid paid cloud accounts.
- **Operational:** the interface needs no training; an admin manages rules and budget from the browser.

Chapter 3: Technology stack

The backend is Java with Spring Boot; the user interface is React. React was chosen over Java UI options (JavaFX, Swing, JSP) because it gives smooth animated charts, live updates and a modern look in any browser and on phones.

Layer	Technology	Why it was chosen
Language (backend)	Java 21	Course language; strong typing; records and modern switch make code short
Framework	Spring Boot 3.5.6	Builds a REST API, scheduling and security with little setup

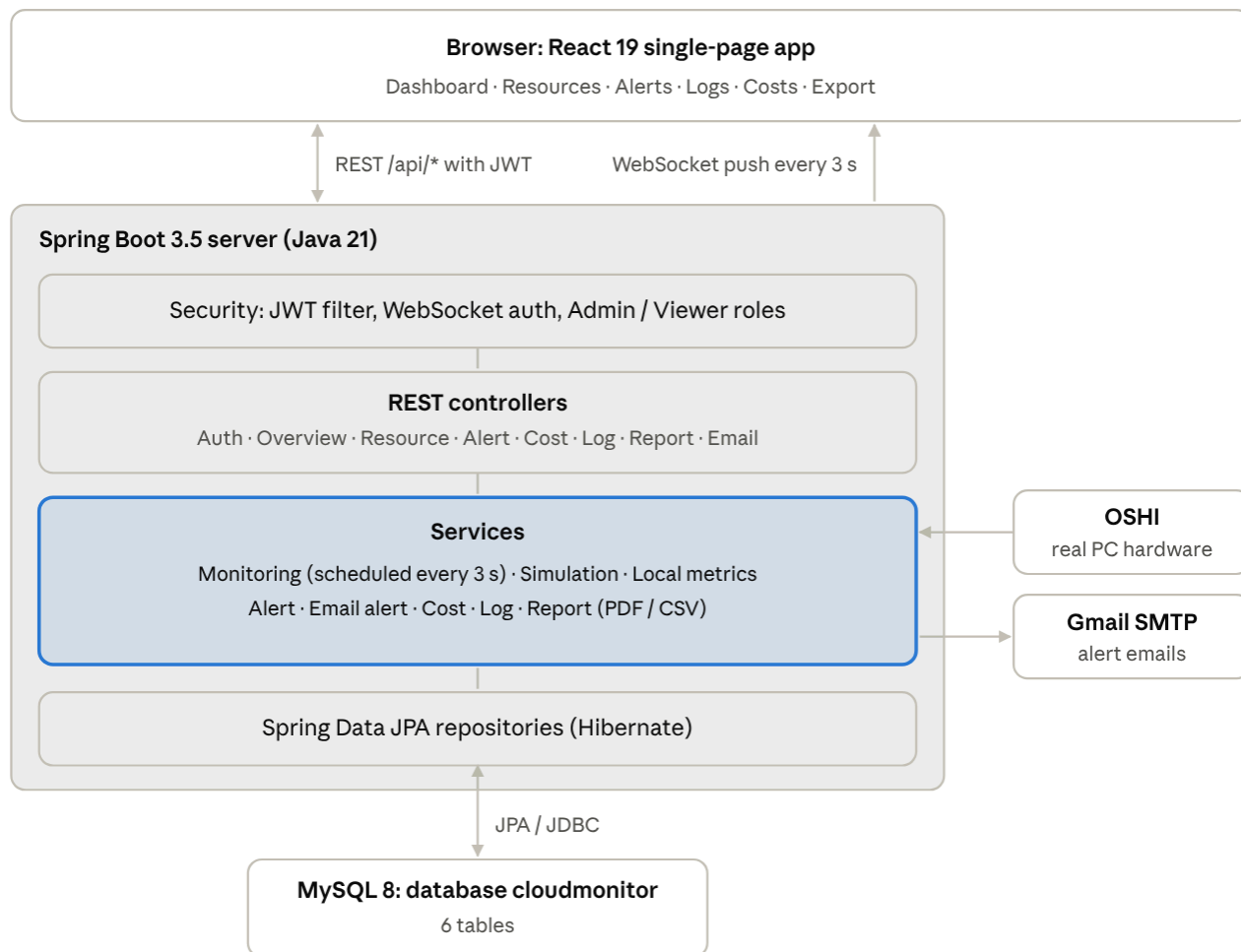
Layer	Technology	Why it was chosen
Live updates	Spring WebSocket + STOMP	Server pushes new data to all browsers every 3 s, no page refresh
Database access	Spring Data JPA, Hibernate 6.6	Java classes map to tables; queries written as method names
Database	MySQL 8.0 (H2 for quick testing)	Free, widely used relational database; data kept after restart
Security	Spring Security, JWT (jjwt 0.12.6), BCrypt	Stateless login tokens; roles checked with @PreAuthorize
System metrics	OSHI 6.6.5	Reads real CPU, RAM, disk, network and uptime of the PC
PDF	OpenPDF 2.0.3	Generates the PDF report in Java
Email	Spring Boot Mail (Gmail SMTP)	Sends alert emails with an App password
Build	Maven	Downloads libraries and builds the backend
Frontend	React 19, Vite 8	Fast single-page app and dev server
Styling	Tailwind CSS 4	Consistent design, dark and light themes
Charts	Recharts 3	Area, bar and donut charts
Animation	framer-motion	Smooth transitions, animated numbers
Other frontend	axios, react-router, @stomp/stompjs, lucide-react icons	API calls, page routing, WebSocket client, icons
Tools	IntelliJ IDEA, Git, GitHub	Development and version control

Chapter 4: System design

CloudPulse uses a three-tier client-server design: a React app in the browser, a Spring Boot server, and a MySQL database. The browser talks to the server in two ways: REST calls for pages and actions, and one WebSocket connection that receives live updates.

4.1 Architecture

One Spring Boot server feeds every browser live and keeps all data in MySQL



System architecture · browser, Spring Boot layers, MySQL, OSHI and Gmail

Requests go down through security, controllers, services and repositories to MySQL; every 3 seconds the services push fresh data back up to all browsers over WebSocket.

4.2 How one update flows (every 3 seconds)

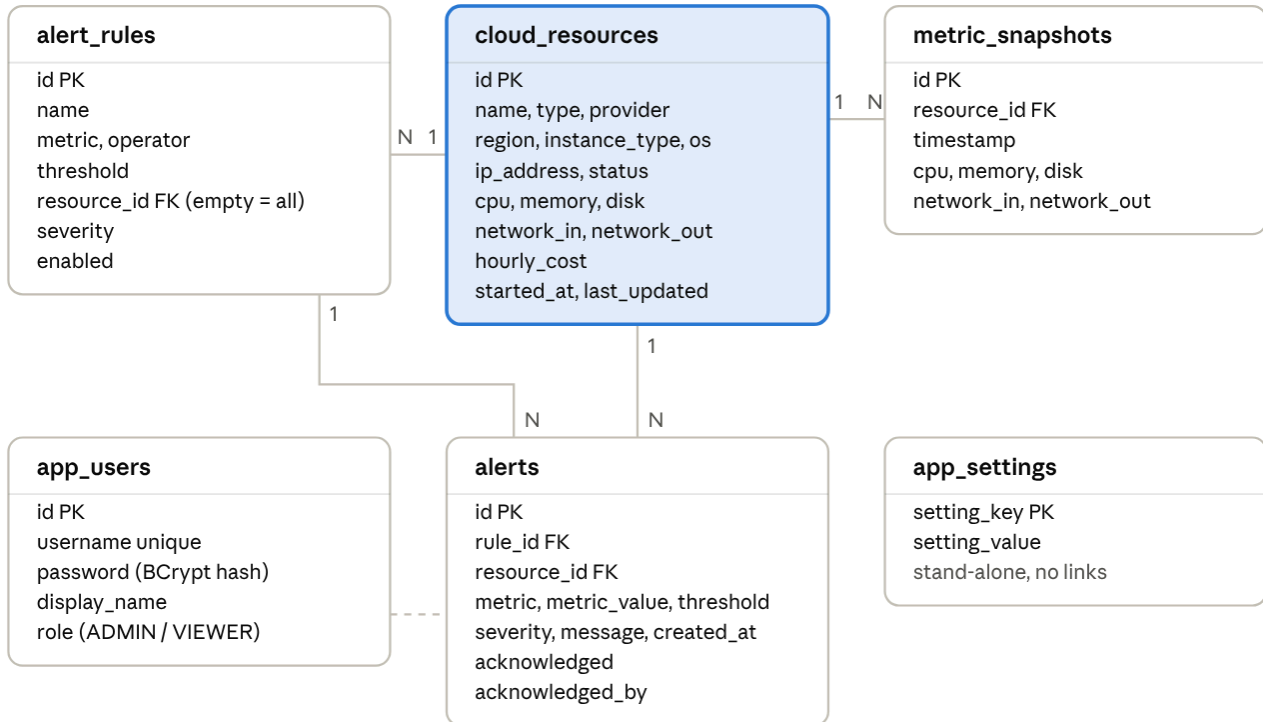
1. A scheduled job in `MonitoringService` runs every 3 seconds.
2. For each running resource it gets new values: `SimulationService` for the 10 cloud servers, `LocalMetricsService` (OSHI) for the real PC.
3. It saves the latest values in `cloud_resources`; every 5th tick (15 s) it also saves a history row in `metric_snapshots`.
4. `AlertService` checks every enabled rule; a breach creates an `alerts` row (at most once per 2 minutes per rule and resource) and, if critical, `EmailAlertService` sends an email in the background.
5. `LogService` writes new log lines for the resources.

6. The server pushes the overview and all resources on `/topic/live`, new alerts on `/topic/alerts` and log lines on `/topic/logs`; every open browser updates at once.

4.3 Database design

The database `cloudmonitor` has six tables, created automatically by Hibernate from the Java entity classes. Tables are linked through ID columns (logical foreign keys).

Every metric, rule and alert is linked to a resource



PK primary key · FK linked by ID · 1 to N one-to-many · dashed line: acknowledged_by holds a username

ER diagram · 6 tables in database cloudmonitor

One resource has many history points, rules and alerts; one rule has many alerts. `app_settings` stores single values such as the budget and email settings.

Table	Purpose	Main columns
<code>app_users</code>	Login accounts	<code>id</code> , <code>username</code> (unique), <code>password</code> (BCrypt hash), <code>display_name</code> , <code>role</code> (ADMIN / VIEWER)
<code>cloud_resources</code>	Each monitored resource and its latest values	<code>id</code> , <code>name</code> , <code>type</code> , <code>provider</code> , <code>region</code> , <code>instance_type</code> , <code>ip_address</code> , <code>os</code> , <code>status</code> , <code>cpu</code> , <code>memory</code> , <code>disk</code> , <code>network_in</code> , <code>network_out</code> , <code>hourly_cost</code> , <code>started_at</code> , <code>last_updated</code>

Table	Purpose	Main columns
metric_snapshots	History points, kept 24 hours	id, resource_id, timestamp, cpu, memory, disk, network_in, network_out (indexed on resource_id + timestamp)
alert_rules	Conditions that raise alerts	id, name, metric, operator (> or <), threshold, resource_id (empty = all), severity, enabled
alerts	Alerts that fired	id, rule_id, resource_id, resource_name, metric, metric_value, threshold, severity, message, created_at, acknowledged, acknowledged_by
app_settings	Settings changed at runtime	setting_key, setting_value (monthly budget, email on/off, recipients, minimum severity)

4.4 REST API

All endpoints except login need the header `Authorization: Bearer <token>`. Endpoints marked Admin return 403 for viewers.

Method	Endpoint	Purpose	Access
POST	/api/auth/login	Log in, returns JWT	Public
GET	/api/auth/me	Current user	All
GET	/api/overview, /api/overview/history?range=	Totals and averaged history	All
GET	/api/resources, /api/resources/{id}	List and detail	All
GET	/api/resources/{id}/metrics? range=, /{id}/alerts	History and alerts of one resource	All
POST	/api/resources/{id}/action	START, STOP or RESTART	Admin
GET	/api/alerts, /api/alerts/active	Alert history, unacknowledged alerts	All

Method	Endpoint	Purpose	Access
POST	/api/alerts/{id}/ack, /api/alerts/ack-all	Acknowledge	All
GET / POST / PUT / DELETE	/api/alerts/rules	Read and manage rules	Read: all, change: Admin
GET / PUT	/api/costs, /api/costs/budget	Cost report, change budget	Read: all, change: Admin
GET	/api/logs, /api/logs/download	Recent log lines, .log file	All
GET	/api/reports/summary.pdf, resources.csv, alerts.csv, costs.csv	Downloads	All
GET / PUT / POST	/api/settings/email, /api/settings/email/test	Email settings and test email	Read: all, change: Admin
WebSocket	/ws (STOMP): /topic/live, /topic/alerts, /topic/logs	Live updates	Token in CONNECT

Chapter 5: Modules and implementation

The backend has 54 Java classes (about 3,100 lines) in seven packages: `config`, `controller`, `dto`, `model`, `repository`, `security` and `service`. The frontend has about 2,800 lines of React code in pages, components, context and helpers.

5.1 Authentication and roles

`AuthController` checks the username and password with Spring Security; passwords are stored as BCrypt hashes. On success `JwtService` creates a signed JWT containing the username and role, valid for 8 hours. `JwtAuthFilter` reads the token on every request, and `StompAuthInterceptor` checks it when the browser opens the WebSocket. Admin-only methods carry `@PreAuthorize("hasRole('ADMIN')")`. Demo users are `admin/admin123` and `viewer/viewer123`.

5.2 Monitoring engine

`MonitoringService.collect()` runs every 3 seconds (`@Scheduled`). It updates every resource, sets its status (Running below 75%, Warning from 75%, Critical from 90% of its worst metric), saves history every 15 seconds and pushes one message to all browsers. History older than 24 hours is deleted every 10 minutes. For charts, history is averaged into at most 90 points.

5.3 Real metrics (OSHI)

`LocalMetricsService` uses OSHI to read the real CPU load (difference between two tick readings), used memory, used disk space across all drives, network bytes per second and the Windows uptime of the computer running the backend. It appears as "CECIL (this PC)". Its values were checked against Windows Task Manager and matched.

5.4 Cloud simulation

`SimulationService` gives each simulated server a profile (base CPU and memory by type: database, VM, container, storage, function). Values follow a daily curve (busier in the afternoon), add random noise, and have a 0.15% chance per tick of a 30 to 90 second load spike above 85% CPU. Disk fills slowly and a "cleanup" resets it near 96%. On first start, 24 hours of history (14,400 points) are generated so charts are not empty.

5.5 Alerts

`AlertService` compares each enabled rule with each resource (for example "CPU > 95 = Critical"). A 2-minute cooldown per rule and resource stops repeats. Alerts are saved, pushed live as pop-ups, written to the log, and handed to the email module. Users acknowledge alerts one by one or all at once; the name of the person is saved.

5.6 Email alerts

`EmailAlertService` sends HTML emails through Gmail SMTP (port 587, TLS) using a 16-character App password read from the environment variables `MAIL_USERNAME` and `MAIL_PASSWORD`. Sending runs on a background thread so the 3-second loop never waits. Admins choose recipients (up to 5) and the minimum severity; a limit of 20 emails per hour prevents flooding.

5.7 Cost module

`CostService` uses the hourly price of each resource. Burn rate = sum of hourly prices of running resources; projected month = burn rate × 730 hours. Spend in the last 24 hours is counted per clock hour from history (a resource is billed for an hour if it was running in it).

Suggestions: run a resource that is 40% or more of the bill only in working hours (saves 50%), and downsize servers whose 24-hour average CPU is under 25%. The budget is saved in `app_settings`.

5.8 Logs

`LogService` keeps the latest 5,000 log lines in memory and streams each new line on `/topic/logs`. Simulated servers write lines that fit their role (nginx requests, PostgreSQL slow queries, Redis saves, model-training epochs), and lines turn into timeouts and errors when a server's CPU is high. The real PC writes a line with its actual readings every 30 seconds.

5.9 Reports

`ReportService` builds a two-page A4 PDF with OpenPDF (key numbers, budget, costs by provider, saving tips, all resources, last 20 alerts) and CSV files that open in Excel (UTF-8 with BOM).

5.10 Frontend

Page	What it shows
Login	Login form with demo account buttons
Dashboard	5 stat cards, utilisation chart, health donut, top CPU users, recent alerts, network chart, providers
Resources	Search, filters, sorting, card and table views, start/stop/restart
Resource detail	Gauges, history charts by range, details, alerts, logs button
Alerts	Active, History, Rules editor, Email settings
Logs	Live terminal-style viewer with filters, search, pause, download
Costs	Burn rate, budget bar, hourly spend chart, provider and type breakdown, suggestions, per-resource table

One WebSocket connection (`LiveContext`) is shared by all pages. The theme (dark default) and login token are remembered in the browser.

Chapter 6: Screenshots

Insert one screenshot per item below (Win + Shift + S on Windows), each with its figure number and caption.

Figure	Screen to capture	What to show
6.1	Login page	Dark theme, demo account buttons
6.2	Dashboard	Live badge, stat cards, charts
6.3	Dashboard, light theme	Theme toggle working
6.4	Resources (cards and table)	Filters and status badges
6.5	CECIL (this PC) detail beside Task Manager	Real metrics matching
6.6	Alerts, Active tab with a pop-up	Live alert
6.7	Alerts, Rules editor	Creating a rule
6.8	Alerts, Email tab + received Gmail	Email alert working
6.9	Costs page	Budget bar and suggestions
6.10	Logs page with Errors filter	Live logs
6.11	PDF report (page 1)	Export
6.12	MySQL: SHOW TABLES and COUNT(*)	Data stored in MySQL
6.13	Viewer login	Admin buttons hidden for viewers
6.14	Mobile view (Chrome DevTools phone mode)	Responsive layout

Chapter 7: Testing

The system was tested manually, module by module, on Windows 11 with Chrome, MySQL 8.0 and Java 21. All 16 test cases passed after the fixes listed in 7.2.

7.1 Test cases

No.	Test	Steps	Expected result	Result
1	Valid login	Log in as admin/admin123	Dashboard opens	Pass

No.	Test	Steps	Expected result	Result
2	Invalid login	Wrong password	"Invalid username or password", no access	Pass
3	Protected API	Call /api/resources without token	401 Unauthorized	Pass
4	Viewer role	Log in as viewer, open a resource	Start/Stop and rule editing not available	Pass
5	Live updates	Watch dashboard for 30 s	Numbers and charts change every 3 s, "Live" badge green	Pass
6	Real PC metrics	Compare "this PC" with Task Manager	Memory within 1%, uptime matches	Pass
7	History ranges	Switch chart to 1 h, 6 h, 24 h	Charts load past data	Pass
8	Stop / start	Admin stops a server, then starts it	Status Stopped, metrics 0; then Running	Pass
9	Alert rule	Leave rules on and wait for a spike	Pop-up and new row in Active alerts	Pass
10	Acknowledge	Click Acknowledge on one alert	Removed from Active, shown in History with user name	Pass
11	Email alert	Turn email on, Send test email	Email received in Gmail inbox	Pass
12	Budget	Change budget to \$2,500	Bar turns red, "over by \$159"	Pass
13	PDF export	Export, Full report	2-page PDF with correct numbers	Pass
14	Logs	Open Logs, filter Errors, search "timed"	Only matching lines, word highlighted	Pass
15	Persistence	Restart backend, count rows in MySQL	Data kept, no re-seeding	Pass
16	Responsive	Open on phone-size screen	Sidebar becomes a menu, layout fits	Pass

7.2 Problems found and fixed

Problem	Cause	Fix
Startup error: table ALERTS not found	Column named <code>value</code> is a reserved word in H2/MySQL	Renamed column to <code>metric_value</code>
"Disk 33%" vs Task Manager "Disk 2%"	Different meaning: space used vs disk activity	Labelled as "Disk used"
Uptime showed backend run time	Used app start time	Read Windows uptime from OSHI
H2ConsoleProperties error after moving to MySQL	Security rule still referred to the H2 console	Replaced it with a plain path rule
150+ alerts in 15 minutes	Spike chance too high (0.6% per tick)	Reduced to 0.15% per tick
Email "Not set up"	Space before variable name in run configuration	Trim names and spaces in code; log a clear status line
Hourly cost bars at :30	Hours cut in UTC, India is UTC+5:30	Cut hours in local time zone

Chapter 8: Limitations, conclusion and future scope

8.1 Limitations

- The 10 cloud resources are simulated; only the local computer is measured for real.
- Logs are kept in memory (last 5,000 lines) and are cleared when the backend restarts.
- Costs use fixed hourly prices, not real provider bills.
- Testing was manual; there are no automated unit tests yet.
- The database password and JWT secret are in the configuration file, acceptable for a demo but not for production.

8.2 Conclusion

CloudPulse meets all eight objectives set in Chapter 1. It shows the health of 11 resources live every 3 seconds, keeps 24 hours of history in MySQL, raises and emails alerts, controls resources by role, estimates costs against a budget, streams logs and exports

reports. Building it covered REST API design, real-time communication with WebSocket, relational database design with JPA, authentication with JWT, scheduled background jobs and a modern React interface.

8.3 Future scope

1. Connect real cloud accounts through the AWS CloudWatch, Azure Monitor and Google Cloud Monitoring APIs.
2. Install a small agent on remote servers so more real machines can be monitored.
3. Store logs in a database or search engine (for example Elasticsearch) instead of memory.
4. Predict problems with simple trend analysis or machine learning (for example "disk full in 3 days").
5. Alerts on more channels: SMS, Slack, Microsoft Teams, Telegram.
6. Automated tests with JUnit and Mockito, and continuous integration on GitHub Actions.
7. Deploy with Docker on a cloud server so it can be opened from anywhere.
8. Move secrets (database password, JWT key) to environment variables or a secrets manager.

8.4 References

1. Spring Boot reference documentation, spring.io/projects/spring-boot
2. Spring Security reference, spring.io/projects/spring-security
3. Spring Framework: WebSocket and STOMP support, docs.spring.io
4. Hibernate ORM documentation, hibernate.org/orm
5. MySQL 8.0 Reference Manual, dev.mysql.com/doc
6. OSHI (Operating System and Hardware Information), github.com/oshi/oshi
7. OpenPDF, github.com/LibrePDF/OpenPDF
8. JSON Web Tokens, RFC 7519, and the jjwt library, github.com/jwt/jwt
9. React documentation, react.dev
10. Recharts documentation, recharts.org
11. Tailwind CSS documentation, tailwindcss.com
12. Gmail SMTP and App passwords, support.google.com/accounts